capital

# Custom DRC Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
**All rights reserved**.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

**RESTRICTED RIGHTS LEGEND 03/97**

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

**Contractor/manufacturer is:**
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

**TRADEMARKS:** The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

**End-User License Agreement:** You can print a copy of the End-User License Agreement from: www.mentor.com/terms_conditions/enduser.cfm.

## Table Of Contents

**Introduction** ..... **2**
Additional Documentation ..... 2
**Interfaces** ..... **3**
IXDRCheck ..... 3
**Exercise** ..... **6**
**Guidance** ..... **8**
**Examples** ..... **9**
**Appendix A – Objects Passed to IXDRCheck** ..... **10**
**Appendix B – IXDRCheck and Validation** ..... **12**
**Appendix C – External Browser Support** ..... **13**

# Introduction

This document describes the process for developing Custom DRC plugins for Capital Logic, Capital Topology, Capital Integrator and Capital HarnessXC. This allows you to develop your own DRC checks on the designs that you create in these applications. If you are **not** interested in developing Custom DRCs, you do not have to read this document. If you are interested in developing Custom DRCs, ensure that you have read and completed the exercises in the PluginDevelopment.pdf document before reading and completing the exercises in this document.

## Additional Documentation

Just to remind you, the documentation for the Java interfaces (the API) can be found within the Capital installation folder in the following location:

```
doc/plugin/api/index.html
```

Moreover, the plugins you develop will be passed in “business” objects as parameters, from which the Capital information you require can be obtained. The relationships between these objects are expressed in a UML diagram.

The UML diagram for the business objects can be found within the Capital installation folder in the following file:

```
doc/plugin/api/BusinessObjectsClassDiagram.png
```

It is important to note that these business objects give you read-only access to the underlying object model used by Capital. You will not be able to use them to modify any object model data. Also, they do not contain graphical information about objects, just the properties and attributes of the objects and their relationship with each other.

The UML diagram for the plugin interfaces can be found within the Capital installation folder in the following file:

```
doc/plugin/api/PluginObjectsClassDiagram.png
```

# Interfaces

## IXDRCheck

In order to build a custom DRC plugin, this is the interface that you will need to implement:

```
public interface IXDRCheck extends IXPlugin
{
      enum Severity { Information, Warning, Error }

      void begin(IXDRCViolationReporter reporter);

      void check(IXDRCViolationReporter reporter, IXObject object);

      void end(IXDRCViolationReporter reporter);

      Severity getDefaultSeverity();

      boolean getDefaultAvailability();
}
```

As you can see `IXDRCheck` extends the `IXPlugin` interface. This means that you will need to implement the `getName()`, `getVersion()` and `getDescription()` methods. Additionally, you will need to implement the following methods:

`begin()` – will be called at the start of the DRC operation on a particular design. You can use this to initialize any data required.

`end()` – will be called at the end of the DRC operation on a particular design. You can use this method to free any data/resources you have acquired. Some DRCs you develop may only be able to detect violations when all objects in the design have been seen.

`getDefaultSeverity()` – will be called to determine what the severity of the check will be when displayed in the DRC check GUI.

`getDefaultAvailability()`– will be called to determine if this check should be enabled or disabled by default when displayed in the DRC check GUI.

`check()`– will be called once for each of the objects that are in the design. So if there are 100 wires, 50 devices and 2 connectors in your Capital Logic design, the `begin()` method will be called once, then the `check()` method will be called 152 times, once for each object, then the `end()` method will be called once. Within the check() method, your code will be able to look at the attributes of the object using the `IXObject` interface. If your custom check is interested in a particular type of object, you can do the following:

```
void check (IXDRCViolationReporter reporter , IXObject object )
{
      if ( object instanceof IXDevice )
      {
            IXDevice device = (IXDevice) object;
            //…
      }
}
```

So, by using casting, the custom DRC you develop can focus on particular types of object. To report an issue, an object is provided to the `begin()`, `check()` and `end()` methods. This object implements the `IXDRCViolationReporter` interface:

```
public interface IXDRCViolationReporter
{
      void report(IXDRCheck.Severity severity,
                  String             message,
                  IXObject ...       involving);
}
```

The first parameter you pass into this method should indicate the severity of the violation. You should pass the following `enum` values from the `IXDRCheck` interface:

`Information`, `Warning` or `Error`.

The next parameter you should pass in is the text message to be displayed to the user. If you need to support internationalization, you may want to obtain this string from a properties file.

After the first two parameters, you can pass in any number of `IXObjects` that have violated your check. Here is an example:

```
void check (IXDRCViolationReporter reporter , IXObject object)
{
      if ( object instanceof IXDevice )
      {
            IXDevice device = (IXDevice) object;
            if(device.getName().startsWith("prototype"))
            {
                  reporter.report(
                      IXDRCViolationReporter.Severity.Information,
                      "device {0} is a prototype and should be removed",
                      device);
            }
      }
}
```

The message string you provide can contain tokens `{0}`, `{1}`, `…{n}`. These tokens will be replaced by the name of the objects you pass in, for example:

```
reporter.report(
   IXDRCViolationReporter.Severity.Information,
   "device {0} should never be directly connected to {1}",
   device1,
   device2);
```

If device1 is called "ECU" and device2 is called "GROUND", this will result in a message:

```
"device ECU should never be directly connected to GROUND"
```

Moreover, in the DRC output, ECU and GROUND will be clickable links to the actual objects on the design.

You should not implement `IXDRCheck` directly; you need to implement one of its child interfaces. These child interfaces do not add any additional methods. However, they do indicate the type of design your custom DRC is meant to check.

To implement a DRC for Logic designs, you will need to implement the `IXLogicDRCheck`. By implementing `IXLogicDRCheck`, the custom plugin will be automatically associated with Capital Logic DRCs.

To implement a DRC for Logic build lists, you will need to implement the `IXLogicBuildListDRCheck`. By implementing `IXLogicBuildListDRCheck`, the custom plugin will be automatically associated with Capital Logic build list DRCs.

To implement a DRC for Topology designs, you will need to implement the `IXTopologyDRCheck`. By implementing `IXTopologyDRCheck`, the custom plugin will be automatically associated with Capital Topology DRCs.

To implement a DRC for Integrator designs, you will need to implement the `IXIntegratorDRCheck`. By implementing `IXIntegratorDRCheck`, the custom plugin will be automatically associated with Capital Integrator DRCs.

To implement a DRC for Harness designs, you will need to implement the `IXHarnessDRCheck`. By implementing `IXHarnessDRCheck`, the custom plugin will be automatically associated with Capital Harness DRCs.

There is nothing stopping you from creating a single custom DRC that can be used in multiple applications. For example, by implementing both the `IXLogicDRCheck` and `IXIntegratorDRCheck` interfaces, the custom plugin you develop will be associated with Logic and Integrator designs.

# Exercise

In this exercise you will develop a custom DRC. The objective of this custom DRC is to check that non slit tube insulations do not pass through junction node. This will be a Capital HarnessXC DRC.

You need to be familiar with Capital HarnessXC to do this exercise (and you need to have a license for Capital HarnessXC).

1) Ensure that you have the necessary Java IDE setup for developing and ANT setup for compiling the plugin. See the section on Java plugins in **PluginDevelopment.pdf** for more information

2) The sample code for this exercise is available at the following location within the Capital installation folder:
doc/plugin/examples/Java/src/com/example/plugin/drc/NonSlitTubeValidatorDRCheck.java

3) Since this is for Capital HarnessXC, the plugin class should implement IXHarnessDRCheck. The sample provided also extends BaseDRCheck so as to reuse the implementation of the methods in IXPlugin

4) The crux of the logic for this DRC is done as a part of the “check” method. “begin” and “end” methods do not have any implementation

5) Since this DRC is specific for bundle insulations, we explicitly do an instance check for the object to be an IXInsulationRun type. An insulation run contains one or more bundle insulations in it

6) To obtain each insulation from this insulation run, we use the “getInsulations” method on IXInsulationRun

7) Since we are interested in slit tubes only, we get the insulation type of each insulation by getting the value of the attribute “InsulationType”

8) If the insulation type is any “tube” type, then we further check if the value of the attribute “Slit” is “true” or “false”. Value “true” indicates a slit tube and not otherwise

9) To get a list of all the nodes in the path of an insulation run, we use the “getNodes” method on IXInsulationRun

10) For each node in the path of the insulation run, we first check if it is a structure node and if yes, then check if it has more than 2 connected bundles (3 or more connected bundles indicate a junction node)

11) If we find such a node, then we report it as an “error” or a “warning” as per the business requirement

12) The details of implementation at each step can be seen in the Java sample provided

13) To build and deploy your plugin, please refer to the section on Java Plugins in the **PluginDevelopment.pdf** document

# Guidance

You may have a list of custom DRCs that you would like to implement. Here are some questions you can ask yourself when determining what custom DRCs you can and cannot do:

- Is the DRC you need to implement in an application that supports custom DRCs?

  Currently, only Capital Logic, Capital Topology, Capital Integrator and Capital HarnessXC can have custom DRCs. You need to ensure that the DRC you want to implement is for one of these applications.

- Do you have access to all the information you need?

  The business objects that you have access to contain the properties and attributes of objects and their relationship with each other. You will not have access to graphical information about objects.

  If you are not sure that you can access the information you need, then you should build a prototype of the custom DRC to investigate.

  As already indicated, only high-level objects are passed into the begin(), check() and end() methods. Other objects are not. You need to interrogate the high-level object to get access to other objects. See Appendix A for more information.

# Examples

At this point, you should have a good idea how to implement this type of plugin. Some examples have been provided to help you. These examples can be found in the following location within the Capital installation:

```
doc/plugin/examples/Java/src/com/example/plugin/drc
```

The following examples are good starting points:

- `ConnectorWireColorValidatorDRCheck.java` – this example ensures that all the wires leading to a connector have different colors.

You can copy and modify the examples to implement your own plugins if you wish. However, you need to check the license and copyright information at the start of the files before you do this.

The examples provided may be different in future releases of Capital. If you do use these examples, therefore, you need make a copy of them (we reserve the right to add, remove or modify these examples).

# Appendix A – Objects Passed to IXDRCheck

Here is a list of the object types that will be passed into the `check()` method. For those objects that are not passed directly into these methods, there are hints on how you can obtain them from the objects that are passed in. The business objects applicable for Capital Integrator are also applicable for Capital Topology.

| Business Object | Logic | Integrator | HarnessXC | Notes |
|---|---|---|---|---|
| IXAbstractConductor | YES | YES | | parent of IXWire, IXNet, IXInterconnect & IXSheild |
| IXAbstractConnector | YES | YES | | parent of IXConnector & IXDeviceConnector |
| IXAbstractDevice | YES | YES | | parent of IXDevice, IXSlot & IXInterconnect Device |
| IXAbstractPin | | | | obtain from IXAbstractDevice, IXAbstractConnector |
| IXAbstractPinList | YES | YES | | parent of IXAbstractDevice |
| IXBackshell | | | | obtain from IXConnector |
| IXBackshellTermination | | | | obtain from IXBackshell.getPins() |
| IXBundle | | YES | Yes | |
| IXBundleRegion | | | | obtain from IXBundle |
| IXCavity | | | | obtain from IXConnector.getPins() |
| IXConnector | YES | YES | Yes | Directly sent by DRC Engine to a DRC. |
| IXDevice | YES | YES | | |
| IXDeviceConnector | | YES | | |
| IXDevicePin | | | | obtain from IXDeviceConnector.getPins() |
| IXGround | YES | YES | | |
| IXHarness | | YES | | Can be obtained from other sub components of a harness. |
| IXHarnessLevel | | | | obtain from IXHarness.getHarnessLevels() |
| IXHole | | | | obtain from IXSlot.getHoles() |
| IXInterconnect | YES | | | |
| IXInterconnectDevice | YES | | | |
| IXMulticore | YES | YES | Yes | Directly sent by DRC Engine to a DRC. |
| IXNet | YES | | | |
| IXNode | | YES | | |
| IXObject | YES | YES | | parent of most objects except IXOption & IXValue |
| IXOption | | | | obtain from IXHarnessLevel.getOptions() |
| IXShield | YES | YES | | |

| | | | | |
|---|---|---|---|---|
| IXSignal | | YES | | |
| IXSlot | | YES | | |
| IXSplice | YES | YES | Yes | Directly sent by DRC Engine to a DRC. |
| IXSplicePin | | | | obtain from IXSplice.getPins() |
| IXValue | | | | obtain from IXObject.getProperties() / getAttributes() |
| IXWire | YES | YES | Yes | Directly sent by DRC Engine to a DRC. |
| | | | | |
| IXWireEnd | | | | Can be obtained from IXWire. |
| IXCavityDetail | | | | Can be obtained from IXCavity. |
| IXAdditionalComponent | | | | Can be obtained from IXNode. |
| IXInsulationRun | | | Yes | Directly sent by DRC Engine to a DRC. |
| IXInsulation | | | | Can be obtained from IXInsulationRun. |
| IXSpotTape | | | Yes | Directly sent by DRC Engine to a DRC. |
| IXBreakoutTape | | | Yes | Directly sent by DRC Engine to a DRC. |
| IXMultilocationComponent | | | Yes | Directly sent by DRC Engine to a DRC. |
| IXNodeComponent | | | | |
| IXOtherComponent | | | | Can be obtained from IXHarness. |
| IXClip | | | Yes | Directly sent by DRC Engine to a DRC. |
| IXGrommet | | | Yes | Directly sent by DRC Engine to a DRC. |
| IXReferenceNode | | YES | Yes | Directly sent by DRC Engine to a DRC. |
| IXStructureNode | | YES | Yes | Directly sent by DRC Engine to a DRC. |
| IXHarnessRegister | | | | Can be obtained from IXHarness. |

You should note that a Capital Integrator(or a Capital Topology) custom DRC does not have access to the devices within a slot. If the slot has a hole with a wire going through it, you will have access to that wire, however. It is possible that a future release of Capital could allow you access to all the devices within slots, but this release does not.

You should also note that Capital Integrator(or a Capital Topology) treats Junction-Boxes and Equipment slots as devices. This will mean that the `check()` method will be called twice for those objects: once as an `IXSlot` and again as an `IXDevice`. But ground slots only get processed once as IXSlot. If your custom DRC is only interested in slots, then using `instanceof IXSlot` will result in you only seeing the slot representation of objects. Similarly, using `instanceof IXDevice` will result in you only seeing the device representation of objects (but not devices within the slots).

# Appendix B – IXDRCheck and Validation

Capital has an object model validation capability. This capability allows the validation of designs to be performed before designs are saved, after designs are loaded and when those designs are changed. These validations are normally very low-level and are used to indicate mistakes that the software may have made. DRCs on the other hand are used to indicate mistakes that the user of the software has made. As a result, these validations are usually developed by Mentor Graphics. However, there is support for you to create your own validations.

A custom DRC, by implementing the `IXValidateable` interface, will be run as a validation. A custom validation must have a default severity of Error. The partial code below indicates, in bold, the changes that you need to make to turn a custom DRC into a custom validation.

```
public class MyValidateableDRCheck implements IXLogicDRCheck,
                                              IXValidateable
{
      // …

      public Severity getDefaultSeverity() {
         return Severity.Error;
      }


}
```

It should be noted that you cannot implement `IXValidateable` on its own. A custom validation must also be a custom DRC. As a result, the custom validation will also be available as a DRC.

# Appendix C – External Browser Support

Yon can now write out HTML links to external web pages. This causes an external HTML browser to be launched. The browser used can be configured in clientprops.xml.

You can use this in a Custom DRC to link to web pages (intranet or internet) that provide additional information about any problem that has been found.

Note: you must configure this based on the platform you are running on. By default it is setup for the Windows platform for Internet Explorer, so you should see the following in clientprops.xml:

```
<externalhyperlinks
     enabled="true"
     handler="externalProgram"
     commandLine="explorer" />
```

You can switch this to use FireFox on Windows by doing the following:

```
<externalhyperlinks
      enabled="true"
      handler="externalProgram"
      commandLine="C:\Program Files\Mozilla Firefox\firefox.exe" />
```

In a Custom DRC, to display the Mentor Graphics web site (for example) in the DRC report:

```
vReporter.report(Severity.Error,
   "Please click <A href=\"http://www.mentor.com\">here</A>.");
```